

Part I. Testing Strategies

For our test plan we will use several different tests to ensure that our fullstack application performs correctly and efficiently across all layers. Our tests will cover all layers starting with unit tests in the backend for all services/managers. We will then move to integration tests using WebApplicationFactory or Testcontainers to make sure the backend api, database and authentication can connect and correctly authenticate requests and return the correct responses. Finally, in the backend we will move to contract/API tests. Here we will make sure that request/response schema, status codes and error formats are correctly passed so the frontend always knows what to expect.

Next we will move to the database for Supabase specific testing. Here we will test Row Level Security policies, to make sure only the correct people can access certain data, roles, and data isolation between users. We will then test migrations to make sure they apply cleanly and that rollbacks work correctly without breaking changes.

Finally we will test the components of our frontend and do end to end tests to make sure that the entire User Experience works correctly. We will do unit and component tests to make sure that functions, hooks and UI components work correctly. Next we will move to integration tests to make sure mocked API interactions are handled correctly and user flows within the application are correct. Finally we will do End-to-End tests to make sure the frontend, backend and db work together using one of the popular End2End tools.

Part II. Test Case Descriptions

BA1.1 Backend API Test 1

BA1.2 This test will ensure all business logic functions correctly.

BA1.3 These tests will use xUnit, moq and FluentAssertions to make sure that all business logic (services and managers) are created correctly during dependency injection, return the correct information for each function call, and provide the right error upon failure for normal cases

BA1.4 Inputs: The inputs for this test will vary depending on the service or manager being tested and what we want to test but only normal data will be included

BA1.5 Outputs: The outputs will vary, we expect the correct output to be provided depending on the input, either the correctly processed data or the correct error code.

BA1.6 Normal

BA1.7 Whitebox

BA1.8 Functional

BA1.9 Unit tests

BA2.1 Backend API Test 2

BA2.2 This test will ensure all business logic functions correctly.

BA2.3 These tests will use xUnit, moq and FluentAssertions to make sure that all business logic (services and managers) are created correctly during dependency injection, return the correct information for each function call, and provide the right error upon failure for abnormal cases

BA2.4 Inputs: The inputs for this test will vary depending on the service or manager being tested but we will be putting abnormal data in to make sure that the business logic can correctly handle it

BA2.5 Outputs: The outputs will vary, we expect the correct output to be provided depending on the input, either the correctly processed data or the correct error code.

BA2.6 Abnormal

BA2.7 Whitebox

BA2.8 Functional

BA2.9 Unit tests

BA3.1 Backend API Test 3

BA3.2 This test will ensure all business logic functions correctly.

BA3.3 These tests will use xUnit, moq and FluentAssertions to make sure that all business logic (services and managers) are created correctly during dependency injection, return the correct information for each function call, and provide the right error upon failure for boundary cases

BA3.4 Inputs: The inputs for this test will be boundary inputs, or anything that is close to not being accepted to make sure that it is handled correctly

BA3.5 Outputs: The outputs will vary, we expect the correct output to be provided depending on the input, either the correctly processed data or the correct error code.

BA3.6 Boundary

BA3.7 Whitebox

BA3.8 Functional

BA3.9 Unit tests

BA4.1 Backend API Test 4

BA4.2 This test will ensure all backend logic integrates correctly

BA4.3 These tests will use WebApplicationFactory and Testcontainers to test the integration of the backend API, database and Authentication to make sure that everything works correctly for normal cases

BA4.4 Inputs: Inputs will vary based on the desired outcome of the test and what part of the integration is being tested

BA4.5 Outputs: Outputs will vary based on the inputs provided. We expect either the correct data is returned or the correct error code to be returned for every test.

BA4.6 Normal

BA4.7 Whitebox

BA4.8 Functional

BA4.9 Integration tests

BA5.1 Backend API Test 5

BA5.2 This test will ensure all backend logic integrates correctly

BA5.3 These tests will use WebApplicationFactory and Testcontainers to test the integration of the backend API, database and Authentication to make sure that everything works correctly for abnormal cases

BA5.4 Inputs: Inputs will vary based on the desired outcome of the test and what part of the integration is being tested

BA5.5 Outputs: Outputs will vary based on the inputs provided. We expect either the correct data is returned or the correct error code to be returned for every test.

BA5.6 Abnormal

BA5.7 Whitebox

BA5.8 Functional

BA5.9 Integration tests

BA6.1 Backend API Test 6

BA6.2 This test will ensure all backend logic integrates correctly

BA6.3 These tests will use WebApplicationFactory and Testcontainers to test the integration of the backend API, database and Authentication to make sure that everything works correctly for boundary cases

BA6.4 Inputs: Inputs will vary based on the desired outcome of the test and what part of the integration is being tested

BA6.5 Outputs: Outputs will vary based on the inputs provided. We expect either the correct data is returned or the correct error code to be returned for every test.

BA6.6 Boundary

BA6.7 Whitebox

BA6.8 Functional

BA6.9 Integration tests

BA7.1 Backend API Contract and API Tests

BA7.2 These tests will ensure the backend api passes the correct responses based on requests

BA7.3 These tests will ensure that the backend api always passes the correct response to the frontend whether it be a response and status code or an error code and error message.

BA7.4 Inputs: the inputs for this test will vary depending on what we want to test (error code, request/response schema)

BA7.5 Outputs: the outputs for this test will vary depending on what we are passing in the inputs, we expect that the output should always be the correct response and status code or error code and error message

BA7.6 normal/abnormal/boundary since these tests will be validating the contracts and api we will need to test normal requests, abnormal requests and boundary requests

BA7.7 Whitebox

BA7.8 Functional

BA7.9 Integration

DB1.1 Database Test 1

DB1.2 This test will test the Row Level Security (RLS) policies in the database

DB1.3 This test will be used to make sure that the Row Level Security policies are functioning correctly and that different roles should only have the information they should have access to and that there is data isolation between users. We will use Supabase CLI, integration test hitting Supabase directly and Supabase web interface

DB1.4 Inputs: inputs will be different role levels either anonymous, authenticated or admin

DB1.5 Outputs: outputs will depend on the RLS policies on the table being tested. Anonymous should not be able to see any data on restricted tables, authenticated users should be only able to see their own data and admins should be able to see all data

DB1.6 Normal

DB1.7 Blackbox

DB1.8 Functional

DB1.9 Integration

DB2.1 Database Test 2

DB2.2 The purpose of this test is to make sure migrations and rollbacks work without causing breaking changes.

DB2.3 This test will be used to make sure that migrations and rollbacks work without causing breaking changes by using Supabase Migrations, Local Supabase Test Database and Supabase CLI

DB2.4 Inputs: the inputs for this test will either be a new migration or the rollback command

DB2.5 Outputs: the outputs will be successful migration or rollback with no errors or migration or rollback with errors. If there are errors since we are testing it locally not on the main database it can be corrected to ensure there are no breaking changes.

DB2.6 Normal

DB2.7 Whitebox

DB2.8 Performance

DB2.9 Integration

UI1.1 Frontend Test 1

UI1.2 This test will test the components of the frontend

UI1.3 For this test we will be testing the frontend functions, hooks and UI components to ensure that they function as expected for normal cases

UI1.4 Inputs: inputs will vary based on the function, hook or UI component being tested

UI1.5 Outputs: The outputs will vary based on the inputs passed, we expect either the correct data to be returned for a function, the correct action to be triggered for a hook or the correct component to be rendered for UI components or else the correct error code and message

UI1.6 Normal

UI1.7 Whitebox
UI1.8 Functional
UI1.9 Unit test

UI2.1 Frontend Test 2

UI2.2 This test will test the components of the frontend

UI2.3 For this test we will be testing the frontend functions, hooks and UI components to ensure that they function as expected for abnormal cases

UI2.4 Inputs: inputs will vary based on the function, hook or UI component being tested

UI2.5 Outputs: The outputs will vary based on the inputs passed, we expect either the correct data to be returned for a function, the correct action to be triggered for a hook or the correct component to be rendered for UI components or else the correct error code and message

UI2.6 Abnormal

UI2.7 Whitebox

UI2.8 Functional

UI2.9 Unit test

UI3.1 Frontend Test 3

UI3.2 This test will test the components of the frontend

UI3.3 For this test we will be testing the frontend functions, hooks and UI components to ensure that they function as expected for boundary cases

UI3.4 Inputs: inputs will vary based on the function, hook or UI component being tested

UI3.5 Outputs: The outputs will vary based on the inputs passed, we expect either the correct data to be returned for a function, the correct action to be triggered for a hook or the correct component to be rendered for UI components or else the correct error code and message

UI3.6 Boundary

UI3.7 Whitebox

UI3.8 Functional

UI3.9 Unit test

UI4.1 Frontend Test 4

UI4.2 These tests will test the user flows within the app as well as the mocked API interactions

UI4.3 For these tests we will test to make sure that the user flows within the app work correctly and that the API interactions work as expected when received for the User flow for normal cases, we will use MSW and React Testing library for these tests

UI4.4 Inputs: the inputs will vary based on the current flow being tested and what we want to test for (either error code/message display, form submission validation, etc.)

UI4.5 Outputs: the outputs will vary depending on the inputs however we expect the correct thing to happen, either an error code and message, form validation error, etc.

UI4.6 Normal

UI4.7 Whitebox
UI4.8 functional
UI4.9 Integration test

UI5.1 Frontend Test 5

UI5.2 These tests will test the user flows within the app as well as the mocked API interactions

UI5.3 For these tests we will test to make sure that the user flows within the app work correctly and that the API interactions work as expected when received for the User flow for abnormal cases, we will use MSW and React Testing library for these tests

UI5.4 Inputs: the inputs will vary based on the current flow being tested and what we want to test for (either error code/message display, form submission validation, etc.)

UI5.5 Outputs: the outputs will vary depending on the inputs however we expect the correct thing to happen, either an error code and message, form validation error, etc.

UI5.6 Abnormal

UI5.7 Whitebox

UI5.8 functional

UI5.9 Integration test

UI6.1 Frontend Test 6

UI6.2 These tests will test the user flows within the app as well as the mocked API interactions

UI6.3 For these tests we will test to make sure that the user flows within the app work correctly and that the API interactions work as expected when received for the User flow for boundary cases, we will use MSW and React Testing library for these tests

UI6.4 Inputs: the inputs will vary based on the current flow being tested and what we want to test for (either error code/message display, form submission validation, etc.)

UI6.5 Outputs: the outputs will vary depending on the inputs however we expect the correct thing to happen, either an error code and message, form validation error, etc.

UI6.6 Boundary

UI6.7 Whitebox

UI6.8 functional

UI6.9 Integration test

FS1.1 Full System Test 1

FS1.2 The purpose of this test is to simulate real users using the application

FS1.3 These tests will test the end to end usability of the application including user sign up, login, performing core application actions, permission based UI changes, offline/online behavior, mobile viewport behavior

FS1.4 Inputs: the inputs for this test will be whatever is needed to test the full stack of the application, either login credentials or sign up information

FS1.5 Outputs/results: the results of this test should demonstrate the fully functioning application

FS1.6 Normal
 FS1.7 Blackbox
 FS1.8 Functional
 FS1.9 Integration

Part III. Test Case Matrix

Test Case ID	Normal/Ab/Bound	Black/Whitebox	Funct/Perform	Unit/Integr.
BA1	Normal	White	Function	Unit
BA2	Abnormal	White	Function	Unit
BA3	Boundary	White	Function	Unit
BA4	Normal	White	Function	Int
BA5	Abnormal	White	Function	Int
BA6	Boundary	White	Function	Int
BA7	Norm/Ab/Bound	White	Function	Int
DB1	Normal	Black	Function	Int
DB2	Normal	White	Perform	Int
UI1	Normal	White	Function	Unit
UI2	Abnormal	White	Function	Unit
UI3	Boundary	White	Function	Unit
UI4	Normal	White	Function	Int
UI5	Abnormal	White	Function	Int
UI6	Boundary	White	Function	Int
FS1	Normal	Black	Function	Int